CENTRO UNIVERSITÁRIO FACENS
CURSO DE PÓS GRADUAÇÃO

REDES NEURAIS ARTIFICIAIS E

DEEP LEARNING / T2025/S2

APRESENTAÇÃO

NOSSAS AULAS SERÃO COM ENCONTROS REMOTOS NAS SEGUINTES DATAS:

17/01, 31/01, 21/02 e 07/03 NO HORÁRIO DAS 8:00 ÁS 17:15; TEREMOS UM PERIODO DE ALMOÇO DE UMA HORA E DOIS INTERVALOS DE 15 MINUTOS.

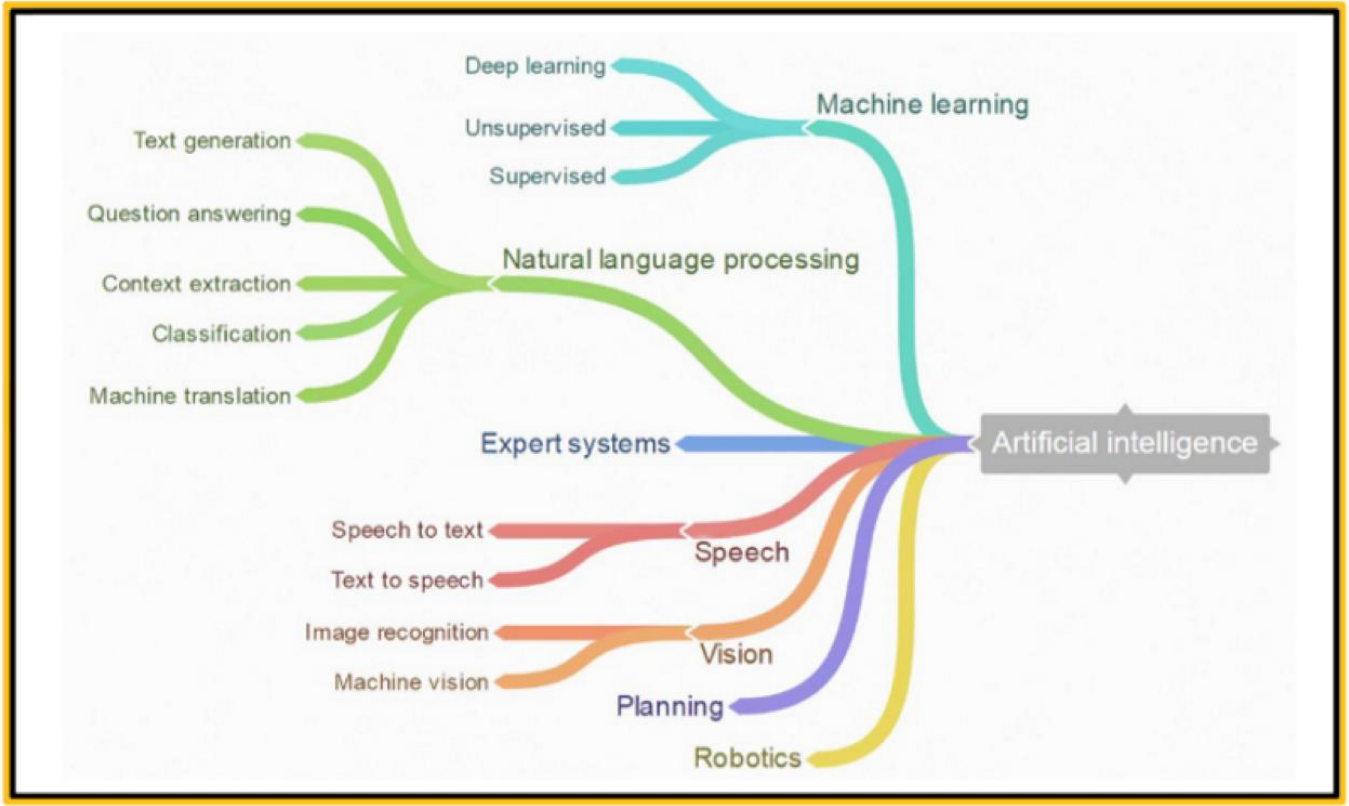
O CONTEÚDO DAS AULAS SERÃO GRAVADOS E O MATERIAL DISPONIBILIZADO PELO CANVAS;

TODOS OS EXERCÍCIOS SERÃO REALIZADOS DURANTE O TEMPO DOS QUATRO ENCONTROS;

TODAS AS ENTREGAS DEVEM SER FEITAS PELO CANVAS;

ALÉM DOS EXERCÍCIOS HAVERÁ UMA ENTREGA FINAL.

REVISÃO



Abordagens para Inteligência Artificial (MARCONI, 2016).

REVISÃO

SCHLEDER, Gabriel R.; FAZZIO, Adalberto. Machine Learning na Física, Química, e Ciência de Materiais: Descoberta e Design de Materiais. Revista Brasileira de Ensino de Física, v. 43, n. Suppl 1, p. e20200407, 2021.

REVISÃO

Característica	Dados Estruturados	Dados Não Estruturados	Dados Semiestruturados
Natureza	Usualmente quantitativa	Usualmente qualitativa	Pode ser quantitativa e qualitativa
Modelo	Predefinido e é difícil alterá-lo	Modelo bem flexível	Tem flexibilidade, mas também possui estrutura
Formato	Número limitado de formato de dados	Grande variedade de formatos de dados	Diversa variedade de formatos
Banco de dados	Bancos baseados em SQL são utilizados	Bancos de dados NoSQL	Bancos de dados relacionais
Pesquisa	Fácil e rápido para localizar e pesquisar esses dados	Como não tem estruturas, é muito difícil procurar esses dados	Pesquisa difícil, mas não tanto quanto nos não estruturados
Análise	Fácil análise	Análise difícil	Análise difícil

REVISÃO

Os conceitos de **dados supervisionados** e **não supervisionados** estão no cerne do Aprendizado de Máquina (Machine Learning), definindo como os algoritmos aprendem a partir das informações fornecidas. A principal diferença reside na presença de **rótulos** (labels) ou respostas corretas nos dados de treinamento

REVISÃO

Dados Supervisionados (Supervised Learning)

No aprendizado supervisionado, o modelo é treinado usando um conjunto de dados que contém tanto as entradas (features) quanto as saídas corretas desejadas (labels/rótulos). Pense nisso como aprender com um professor: o computador recebe exemplos rotulados, prevê a saída e ajusta-se com base no erro.

- Característica: Dados rotulados (Input + Output).
- Objetivo: Mapear uma entrada para uma saída (previsão).

Tipos de Tarefa:

- Classificação: Prever uma categoria ou classe (ex: Spam/Não Spam, Gato/Cachorro).
- Regressão: Prever um valor numérico contínuo (ex: preço de uma casa, previsão de temperatura).

Exemplos de Uso: Reconhecimento de imagem, detecção de fraude bancária, diagnóstico médico.

REVISÃO

Dados Não Supervisionados (Unsupervised Learning)

No aprendizado não supervisionado, o modelo trabalha com dados que não possuem rótulos ou respostas corretas pré-definidas. O algoritmo deve explorar os dados por conta própria para identificar estruturas ocultas, padrões ou agrupamentos.

- Característica: Dados não rotulados (Apenas Input).
- Objetivo: Encontrar estruturas, padrões ocultos ou agrupamentos.

Tipos de Tarefa:

- Agrupamento (Clustering): Agrupar dados similares (ex: K-means).
- Associação: Encontrar regras que descrevem grandes porções dos dados.
- Redução de Dimensionalidade: Simplificar dados complexos sem perder informações essenciais.

Exemplos de Uso: Segmentação de clientes para marketing, sistemas de recomendação, detecção de anomalias.

REVISÃO

Paciente	Idade	IMC	Medicamentos	Quedas	Doenças Crônicas	Visitas ao Posto
1	85	22.5	3	1	2	5
2	78		5	0	1	3
3	90	26.7	4	2		7
4	72	29.4	2	-1	1	2
5	88	31.0	6	3	3	
6		27.3	3	0	2	4
7	82	24.8	4		1	6
8	91	33.5		2	2	9
9	150	28.6	3	0	2	3
10	86	25.4	4	2	3	6

REVISÃO

Machine Learning é uma subárea da Inteligência Artificial que desenvolve modelos matemáticos capazes de aprender padrões a partir de dados, permitindo realizar previsões, classificações ou tomadas de decisão sem programação explícita de regras.

REVISÃO

Aprendizado por Reforço
Aprendizado Supervisionado
Aprendizado Não Supervisionado

REVISÃO

Os modelos supervisionados são treinados com pares de entrada-saída para aprender a prever ou classificar novas entradas, enquanto os modelos não supervisionados exploram a estrutura dos dados sem a necessidade de rótulos ou saídas conhecidas.

Supervisionado:

Em modelos supervisionados, o algoritmo é treinado em um conjunto de dados que inclui tanto os inputs (características ou atributos) quanto os outputs (rótulos ou variáveis dependentes) correspondentes.

O objetivo é aprender a relação entre os inputs e os outputs, de modo que, quando apresentado com novos inputs, o modelo possa prever ou classificar os outputs com base no que foi aprendido durante o treinamento.

Exemplos de modelos supervisionados incluem regressão linear, árvores de decisão, random forest, SVM (Support Vector Machines), entre outros.

REVISÃO

Um exemplo de dados supervisionados seria um conjunto de dados de previsão de preços de imóveis. Neste conjunto de dados, cada instância (linha) representa um imóvel e é composta por características como área, número de quartos, localização, etc. Além disso, cada instância também inclui o preço de venda do imóvel, que é a variável que queremos prever.

Portanto, neste exemplo:

- As características do imóvel (área, número de quartos, localização, etc.) são os atributos de entrada, também chamados de features ou variáveis independentes.
- O preço de venda do imóvel é o atributo de saída, também conhecido como variável dependente ou rótulo.

REVISÃO

Não supervisionado:

Em modelos não supervisionados, o algoritmo é treinado em um conjunto de dados que contém apenas os inputs, sem os correspondentes outputs ou rótulos. O objetivo é explorar a estrutura ou padrões nos dados, sem guia externo sobre o que procurar. O algoritmo tenta encontrar padrões naturais ou agrupamentos nos dados. Não há uma variável alvo específica a ser prevista ou otimizada durante o treinamento.

Os modelos não supervisionados utilizam algoritmos como: k-means para clustering, análise de componentes principais (PCA), algoritmos de redução de dimensionalidade, entre outros.

Um exemplo de dados não supervisionados seria um conjunto de dados de segmentação de clientes para uma empresa de varejo. Neste conjunto de dados, cada instância representa um cliente e é composta por características como idade, gênero, renda, histórico de compras, entre outros.

APRENDIZADO POR REFORÇO

No aprendizado por reforço, o algoritmo aprende por meio da interação com um ambiente. Diferentemente dos modelos supervisionados, não há um conjunto fixo de pares entrada-saída previamente rotulados. Em vez disso, o agente toma decisões, observa as consequências dessas decisões e recebe recompensas ou penalidades.

O objetivo é aprender uma estratégia (política) que maximize a recompensa acumulada ao longo do tempo. Nesse contexto, o modelo aprende por tentativa e erro, ajustando suas ações com base no retorno obtido.

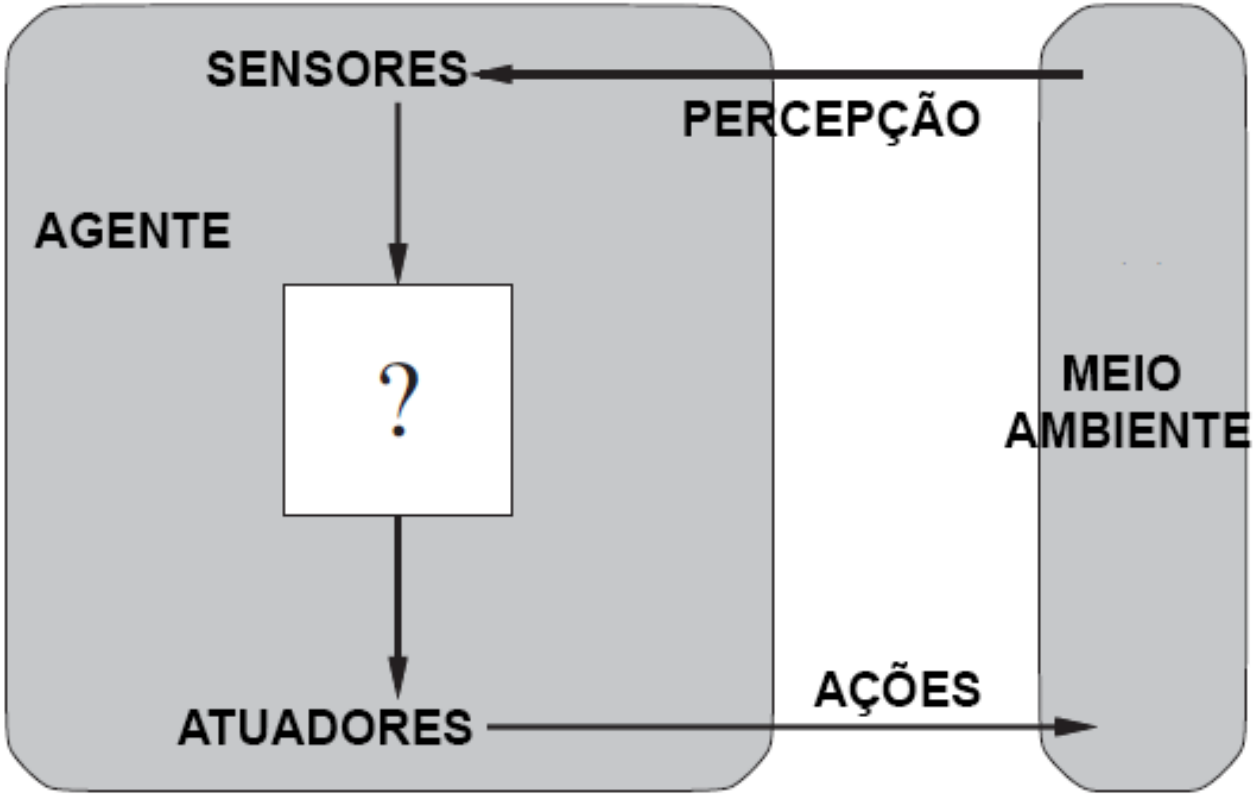
REVISÃO

Componentes principais do aprendizado por reforço

- Agente → quem toma decisões
- Ambiente → onde as ações ocorrem
- Estado (state) → situação atual do ambiente
- Ação (action) → decisão tomada pelo agente
- Recompensa (reward) → feedback numérico após a ação
- Política (policy) → estratégia que define qual ação tomar em cada estado

Agentes com reflexo simples. (RUSSEL; NORVIG, 2003).

REVISÃO



Agentes com reflexo simples. (RUSSEL; NORVIG, 2003).

REVISÃO

Exemplo prático

- Um exemplo clássico é um sistema que aprende a jogar um jogo.
- O agente observa o estado do jogo.
- Escolhe uma ação (movimento).
- O ambiente responde.
- O agente recebe uma recompensa (ganhar pontos ou perder vida).
- Com o tempo, aprende quais ações levam a melhores resultados.

Agentes com reflexo simples. (RUSSEL; NORVIG, 2003).

REVISÃO

Critério	Supervisionado	Não Supervisionado	Aprendizado por Reforço
Tipo de dado	Dados rotulados (X, y)	Apenas dados de entrada (X)	Interação com ambiente
Existe variável alvo?	Sim	Não	Não diretamente (há recompensa)
Objetivo principal	Prever ou classificar	Descobrir padrões ou estrutura	Maximizar recompensa acumulada
Forma de aprendizado	Minimização do erro entre previsão e rótulo	Otimização de estrutura interna dos dados	Tentativa e erro com feedback do ambiente
Feedback recebido	Erro explícito (loss)	Nenhum rótulo externo	Recompensa ou penalidade
Tipo de problema típico	Regressão e Classificação	Clustering e Redução de Dimensão	Tomada de decisão sequencial
Exemplo prático	Previsão de preço de imóveis	Agrupamento de clientes	Agente aprendendo a jogar
Algoritmos comuns	Regressão Linear, SVM, Random Forest, Redes Neurais	K-means, PCA, DBSCAN, Autoencoders	Q-Learning, DQN, PPO, Actor-Critic
Função otimizada	Função de perda (MSE, Cross-Entropy)	Variância, densidade ou distância	Recompensa acumulada esperada
Aplicações típicas	Diagnóstico médico, previsão de demanda	Segmentação de mercado, análise exploratória	Robótica, jogos, veículos autônomos

EXEMPLO DADO NÃO SUPERVISIONADO

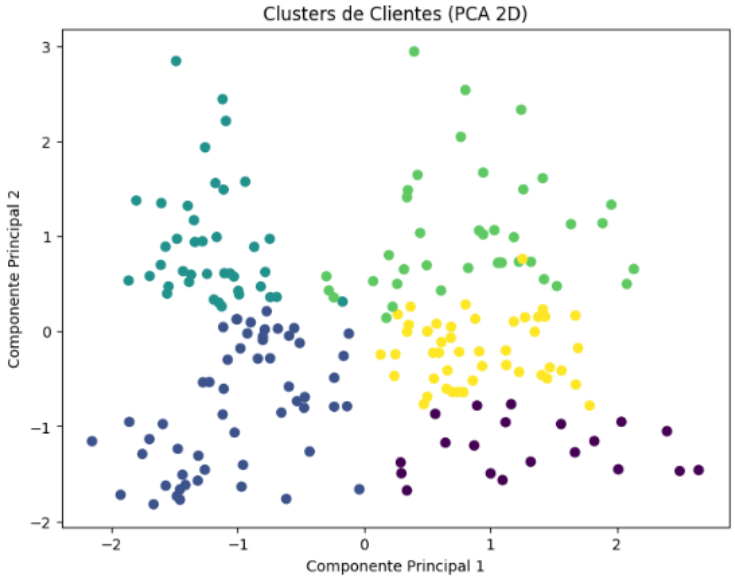
Um shopping center quer aumentar a taxa de conversão de campanhas. A equipe tem dados básicos de clientes (idade, renda anual e “spending score”) e precisa segmentar perfis de compra para criar campanhas diferentes (ex.: “alta renda e alto gasto”, “baixa renda e alto gasto”, etc.).Como não existe rótulo dizendo “cliente do tipo A/B/C”, a tarefa é não supervisionada: clustering.

```python
# ============================
# CENÁRIO 1: Segmentação de clientes com K-means
# ============================

from google.colab import files
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# 1) Upload do CSV
uploaded = files.upload()  # selecione Mall_Customers.csv

# 2) Descobrir nome do arquivo enviado
csv_name = [name for name in uploaded.keys() if name.lower().endswith(".csv")][0]
df = pd.read_csv(csv_name)

print(df.head())

# 3) Selecionar features para clustering
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].copy()

# 4) Padronizar
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 5) K-means
k = 5
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X_scaled)

print("\nResumo por cluster (médias):")
print(df.groupby("cluster")[features].mean().round(2))

# 6) Visualização em 2D com PCA
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["cluster"])
plt.title("Clusters de Clientes (PCA 2D)")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.show()
```

Escolher arquivos Mall_Customers.csv

Mall_Customers.csv(text/csv) - 3981 bytes, last modified: 21/02/2026 - 100% done

```
Saving Mall_Customers.csv to Mall_Customers.csv
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Resumo por cluster (médias):
           Age  Annual Income (k$)  Spending Score (1-100)
cluster
0        46.25               26.75                   18.35
1        25.19               41.09                   62.24
2        32.88               86.10                   81.53
3        39.87               86.10                   19.36
4        55.64               54.38                   48.85
```



EXEMPLO

Um banco quer um “radar” de transações suspeitas. Em fraude real, muitas vezes você não tem rótulo confiável (ou ele chega tarde). Então o time implementa um modelo não supervisionado para detectar anomalias: marcar transações “fora do padrão” para revisão humana.

```python
# ============================
# CENÁRIO 2: Detecção de anomalias com Isolation Forest
# Arquivo esperado: creditcard.csv
# ============================

from google.colab import files
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report, confusion_matrix

# 1) Upload do CSV
uploaded = files.upload()  # selecione creditcard.csv

# 2) Descobrir nome do arquivo enviado
csv_name = [name for name in uploaded.keys() if name.lower().endswith(".csv")][0]
df = pd.read_csv(csv_name)

print(df.head())

```

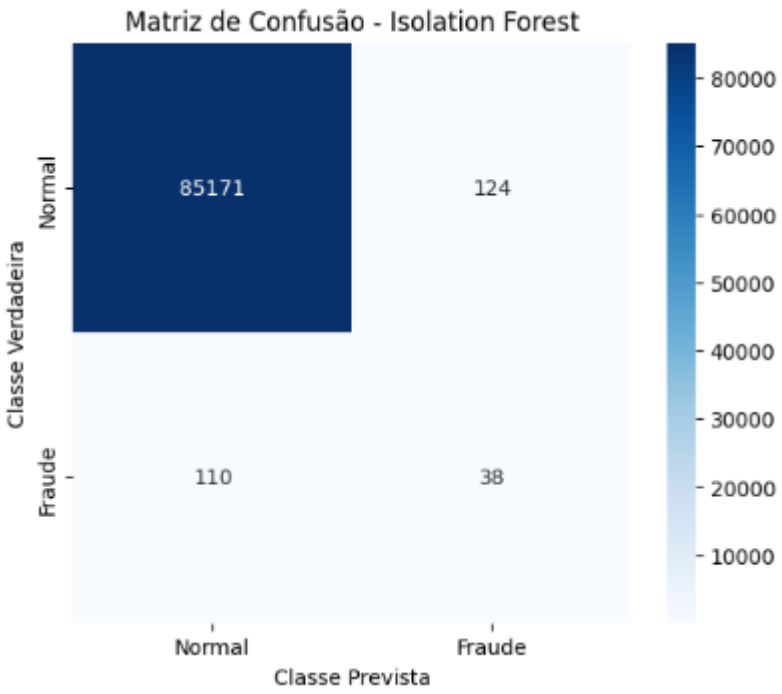
Escolher arquivos creditcard.csv

creditcard.csv(text/csv) - 150828752 bytes, last modified: 21/02/2026 - 100% done

```
Saving creditcard.csv to creditcard (1).csv
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
```

EXEMPLO

```python
# 3) Carrega os dados da pasta
data_path = "./creditcard.csv"
df = pd.read_csv(data_path)

# 4) Separar X e y (y só para validação posterior)
X = df.drop(columns=["Class"]).copy()
y = df["Class"].copy()

# 5) Padronizar
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 6) Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

# 7) Isolation Forest
iso = IsolationForest(
    n_estimators=300,
    contamination=0.002,
    random_state=42
)
iso.fit(X_train)

# 8) Predição
pred_raw = iso.predict(X_test)
y_pred = np.where(pred_raw == -1, 1, 0)

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Calcular matriz
cm = confusion_matrix(y_test, y_pred)

# Criar visualização
plt.figure(figsize=(6,5))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=['Normal', 'Fraude'],
            yticklabels=['Normal', 'Fraude'])

plt.title("Matriz de Confusão - Isolation Forest")
plt.xlabel("Classe Prevista")
plt.ylabel("Classe Verdadeira")
plt.show()
```



EXEMPLO

Uma avenida com cruzamento semaforizado sofre com congestionamento variável: em alguns horários, o fluxo principal aumenta; em outros, a via secundária fica crítica. O modo tradicional usa tempos fixos (ex.: 30s verde, 5s amarelo), mas isso é ineficiente porque:

O tráfego é estocástico (muda por eventos, clima, horários), a decisão ótima depende do estado atual (fila), uma decisão ruim agora “propaga” fila para os próximos minutos.

Esse é um problema típico de **controle sequencial**. Em aprendizado por reforço, o agente aprende a escolher o tempo de verde de forma adaptativa para minimizar filas (ou maximizar vazão).

EXEMPLO

Como isso vira Aprendizado por Reforço

Agente: controlador do semáforo

Ambiente: dinâmica de chegada e saída de carros

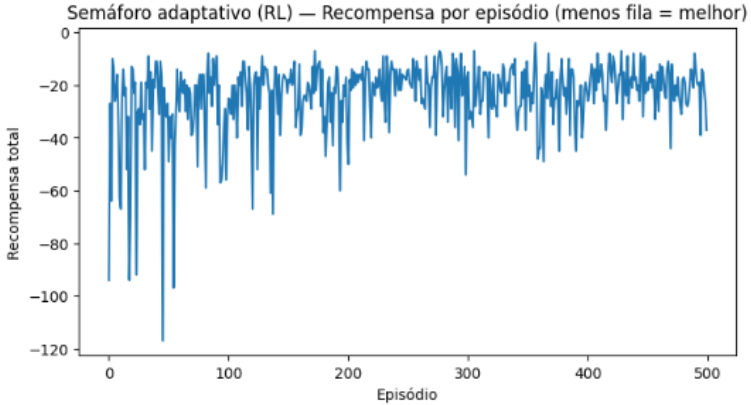
Estado: tamanho da fila atual (discretizada)

Ações: escolher “verde curto”, “médio” ou “longo”

Recompensa: negativa da fila (quanto menor a fila, melhor)

Objetivo: reduzir o congestionamento ao longo do tempo

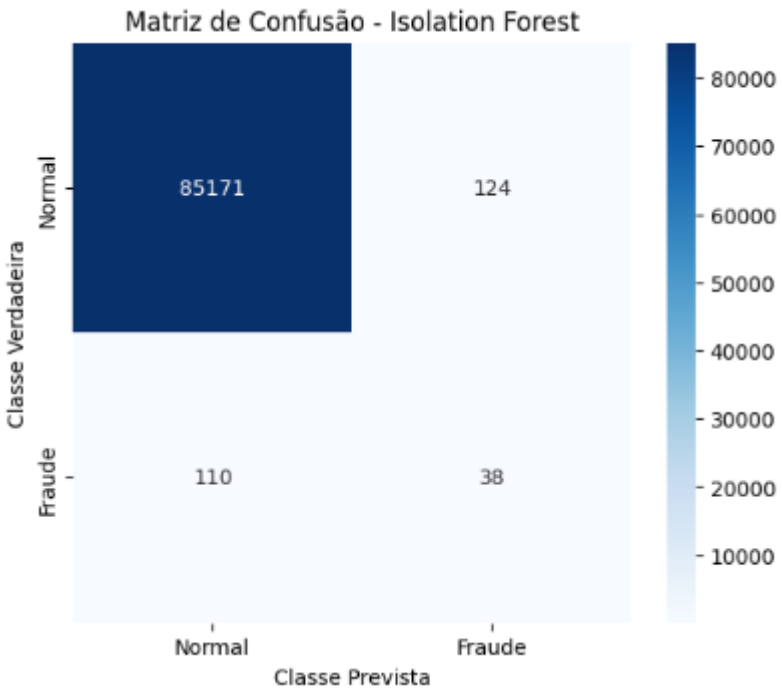
```python
import numpy as np
import matplotlib.pyplot as plt

MAX_QUEUE = 30
ACTIONS = 3  # 0 curto, 1 médio, 2 longo

Q = np.zeros((MAX_QUEUE + 1, ACTIONS), dtype=float)

alpha = 0.10
gamma = 0.90
epsilon = 0.10
episodes = 500

rng = np.random.default_rng(42)
episode_rewards = []

for ep in range(episodes):
    queue = 10
    total_reward = 0.0

    for _ in range(200):  # passos no episódio (simula vários ciclos)
        # Epsilon-greedy
        if rng.random() < epsilon:
            action = int(rng.integers(0, ACTIONS))
        else:
            action = int(np.argmax(Q[queue]))

        # Chegadas aleatórias de carros (varia conforme cenário)
        arriving = int(rng.integers(0, 6))  # 0..5 carros chegam

        # Saída depende da ação (tempo de verde)
        if action == 0:
            leaving = 2
        elif action == 1:
            leaving = 4
        else:
            leaving = 6

        queue_next = queue + arriving - leaving
        queue_next = int(np.clip(queue_next, 0, MAX_QUEUE))

        # Recompensa: queremos fila pequena
        reward = -queue_next
        total_reward += reward

        # Atualização Q
        best_next = np.max(Q[queue_next])
        Q[queue, action] += alpha * (reward + gamma * best_next - Q[queue, action])

        queue = queue_next

    episode_rewards.append(total_reward)

plt.figure(figsize=(8,4))
plt.plot(episode_rewards)
plt.title("Semáforo adaptativo (RL) — Recompensa por episódio (menos fila = melhor)")
plt.xlabel("Episódio")
plt.ylabel("Recompensa total")
plt.show()

# Política aprendida: para cada tamanho de fila, qual ação o agente prefere
policy = np.argmax(Q, axis=1)
print("Fila -> Ação preferida (0 curto, 1 médio, 2 longo)")
for q in [0, 5, 10, 15, 20, 25, 30]:
    print(f"{q:>2} -> {policy[q]}")
```




```
Fila -> Ação preferida (0 curto, 1 médio, 2 longo)
 0 -> 2
 5 -> 2
10 -> 2
15 -> 0
20 -> 0
25 -> 0
30 -> 0
```

EXEMPLO

```python
# 3) Carrega os dados da pasta
data_path = "./creditcard.csv"
df = pd.read_csv(data_path)

# 4) Separar X e y (y só para validação posterior)
X = df.drop(columns=["Class"]).copy()
y = df["Class"].copy()

# 5) Padronizar
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 6) Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

# 7) Isolation Forest
iso = IsolationForest(
    n_estimators=300,
    contamination=0.002,
    random_state=42
)
iso.fit(X_train)

# 8) Predição
pred_raw = iso.predict(X_test)
y_pred = np.where(pred_raw == -1, 1, 0)

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Calcular matriz
cm = confusion_matrix(y_test, y_pred)

# Criar visualização
plt.figure(figsize=(6,5))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=['Normal', 'Fraude'],
            yticklabels=['Normal', 'Fraude'])

plt.title("Matriz de Confusão - Isolation Forest")
plt.xlabel("Classe Prevista")
plt.ylabel("Classe Verdadeira")
plt.show()
```

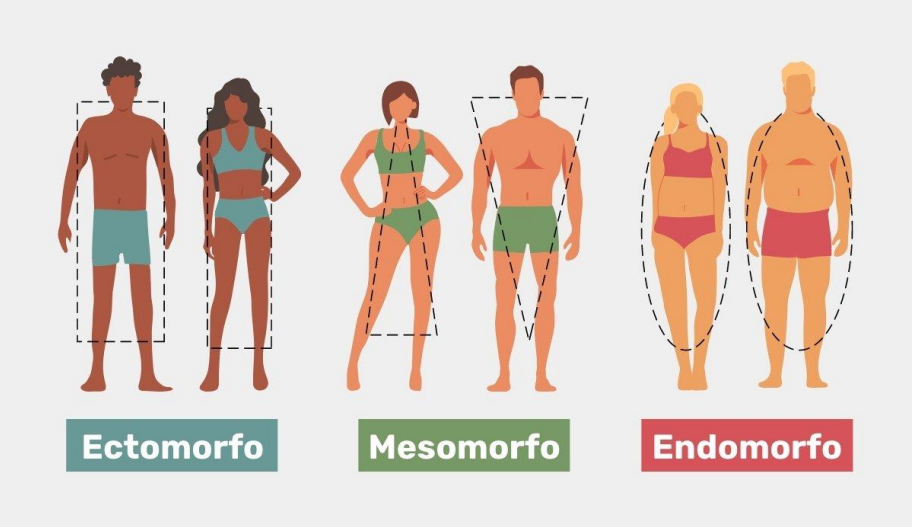


REVISÃO

Esta seção aborda um dos pontos mais críticos onde projetos de Data Science falham: **confiar na métrica errada ou em uma avaliação mal estruturada**, resultando em modelos que parecem ótimos no papel, mas fracassam no mundo real.

Avaliação de modelos

- Holdout vs Cross-validation
- Métricas:
  - Accuracy (quando NÃO usar)
  - Precision / Recall
  - F1-score
  - ROC-AUC
- Matriz de confusão como ferramenta de decisão
- Threshold ≠ 0.5 (impacto organizacional)



REVISÃO

Holdout

Dividimos os dados uma única vez:

- 70% treino
- 30% teste
- O modelo é treinado no treino e avaliado no teste.

Vantagens:

- Simples
- Rápido
- Fácil de implementar

Problema:

- O resultado pode depender muito de **como os dados foram divididos**.
- Se a divisão for “sortuda”, a métrica pode parecer melhor do que realmente é.

REVISÃO

Cross-Validation (Validação Cruzada)

Dividimos os dados em **k partes (k-fold)**: Exemplo: k = 5, Treina em 4 partes, Testa em 1 e repete 5 vezes

Faz a média das métricas

Vantagens:

- Avaliação mais robusta
- Menos dependente do acaso
- Melhor estimativa de generalização

Quando usar:

- Base pequena
- Projetos acadêmicos
- Seleção de modelo

REVISÃO

Acurácia

$$Acurácia = \frac{VP + VN}{Total}$$

O que significa?

Percentual total de acertos.

Quando usar?

Dados equilibrados

Quando erros têm o mesmo peso

Problema:

Se 95% dos dados forem negativos, um modelo que sempre prevê “negativo” terá 95% de acurácia — mesmo sendo inútil.

Por isso não é suficiente em muitos casos.

REVISÃO

Precisão (Precision)

$$Precisão = \frac{VP}{VP + FP}$$

O que significa?

Entre tudo que o modelo disse ser positivo, quantos realmente eram positivos?

Interpretação prática:

"Quando o modelo acusa algo, ele está certo?"

Quando é importante?

Detecção de fraude

Diagnóstico médico

Spam

Você quer evitar **falsos positivos**.

REVISÃO

Recall (Sensibilidade)

$$Recall = \frac{VP}{VP + FN}$$

O que significa?

Entre todos os positivos reais, quantos o modelo conseguiu identificar?

Interpretação prática:

"O modelo consegue encontrar todos os casos importantes?"

Quando é crítico?

Câncer

Fraude

Segurança

Você quer evitar **falsos negativos**.

REVISÃO

F1-Score

$$F1 = 2 \times \frac{Precisão \times Recall}{Precisão + Recall}$$

É a média harmônica entre Precisão e Recall.

O que significa?

Equilíbrio entre:

Não acusar errado (precisão)

Não deixar passar casos importantes (recall)

Quando usar?

Dados desbalanceados

Quando ambos FP e FN são importantes

REVISÃO

ROC = Receiver Operating Characteristic

AUC = Area Under the Curve

O modelo gera probabilidades.

A curva ROC mede:

Taxa de Verdadeiro Positivo (Recall) vs Taxa de Falso Positivo

AUC varia entre:

0.5 → Aleatório

1.0 → Perfeito

Vantagem:

Independe do threshold fixo.

Muito usada em:

Crédito; Medicina; Risco financeiro

REVISÃO

Em modelos de Machine Learning (classificação binária), o **Threshold (limiar) ≠ 0.5** significa que o ponto de corte padrão de 50% para decidir entre a Classe A ou Classe B foi alterado para maximizar o desempenho do modelo em cenários específicos, como dados desequilibrados. Se a probabilidade da previsão for > 0.5, é positivo (1); caso contrário, é negativo (0).

Ajustar esse valor (para, por exemplo, 0.3 ou 0.7) permite ao modelo ser mais ou menos "rígido" na classificação, alterando o equilíbrio entre Precisão (quantos dos classificados como positivos eram realmente?) e Recall (quantos dos positivos reais foram detectados?)

MODELAGEM PREDITIVA E MACHINE LEARNING

Overfitting e underfitting (ponte para redes neurais)

- Curvas de aprendizado
- Complexidade do modelo
- Bias–variance tradeoff
- Regularização (conceitual)

REVISÃO

Overfitting (Sobreajuste)

O modelo aprende **demais**, inclusive o ruído.

Características:

- Erro muito baixo no treino
- Erro alto no teste
- Memoriza os dados de treino

Exemplo:

- Árvore de decisão extremamente profunda.

Intuição:

- O modelo decorou os dados.

REVISÃO

Underfitting (Subajuste)

O modelo é **simples demais** para capturar o padrão real dos dados.

Características:

- Erro alto no treino
- Erro alto no teste
- Não aprende o padrão real

Exemplo:

- Tentar ajustar uma linha reta para dados claramente curvos.

Intuição:

- O modelo é limitado demais.

DÚVIDAS ou PERGUNTAS?

REDES NEURAIS ARTIFICIAIS

As Redes Neurais Artificiais (RNAs), também conhecidas como redes neurais ou ANN (Artificial Neural Networks, em inglês), são um modelo computacional e uma técnica de aprendizado de máquina inspirados na estrutura e no funcionamento do cérebro humano. Elas são usadas para resolver uma variedade de problemas complexos, incluindo classificação de imagens, reconhecimento de fala, previsão de séries temporais, tradução automática e muito mais.

HISTÓRIA DAS RNAs

Neurônios Artificiais Iniciais (Décadas de 1940-1950): O matemático e lógico Warren McCulloch e o neurofisiologista Walter Pitts desenvolveram os primeiros modelos de neurônios artificiais em 1943.

Perceptrons (Década de 1950): O psicólogo e cientista da computação Frank Rosenblatt desenvolveu o conceito de "perceptron" na década de 1950. Os perceptrons eram modelos de neurônios artificiais capazes de aprender pesos de conexão com base em exemplos.

Inverno da IA (Décadas de 1970-1980): Durante as décadas de 1970 e 1980, houve um período de desinteresse em RNAs, conhecido como o "inverno da IA", devido a limitações percebidas nas capacidades das RNAs e avanços limitados na área de aprendizado de máquina.

HISTÓRIA DAS RNAs

Ressurgimento das RNAs (Décadas de 1980-1990): O ressurgimento das RNAs ocorreu com o desenvolvimento de novas arquiteturas, como as Redes Neurais Feedforward Multicamadas (MLP), que incorporam camadas ocultas. Isso permitiu que as RNAs resolvessem problemas mais complexos, incluindo aqueles que eram previamente considerados insolúveis.

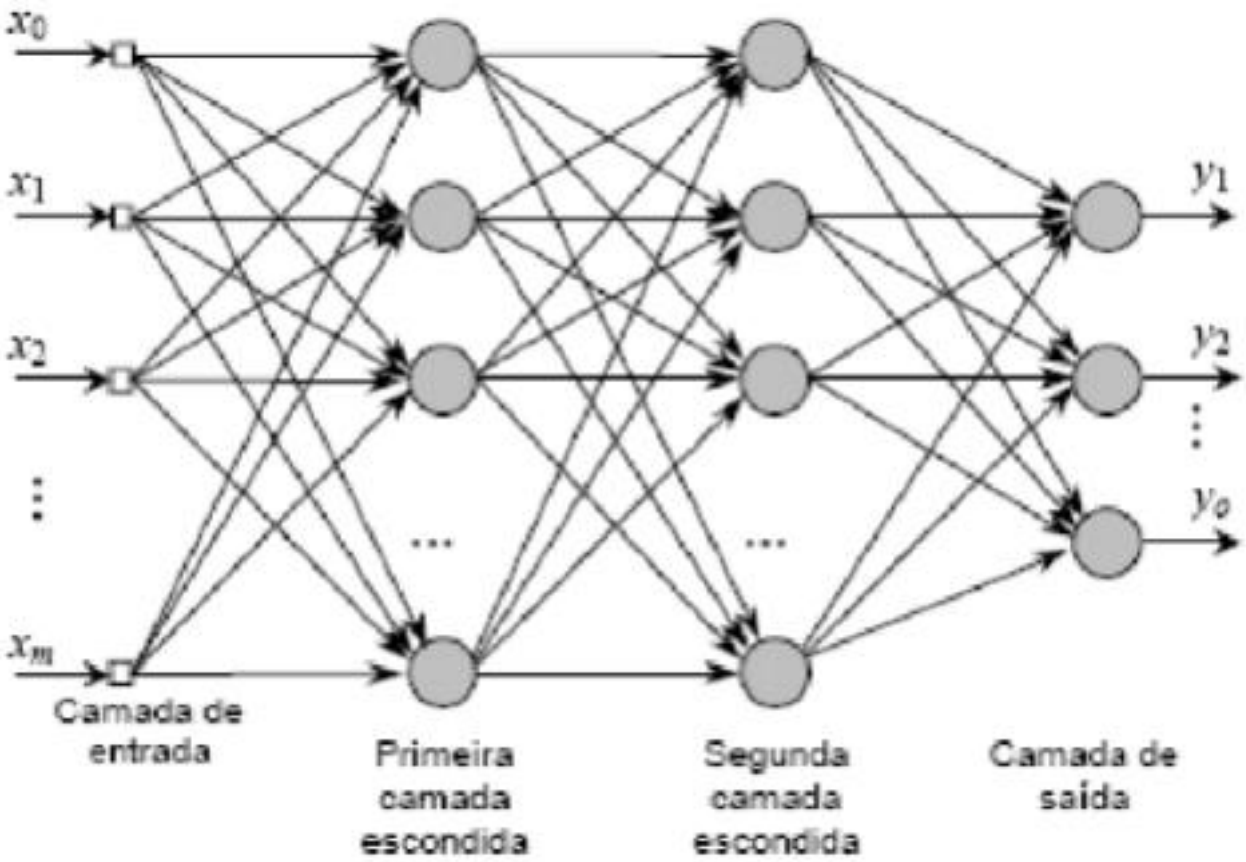
Aprendizado Profundo (Década de 2000 em Diante): O aprendizado profundo (deep learning) é um subcampo das RNAs que se concentra em redes neurais profundas com muitas camadas ocultas. A partir da década de 2000, houve um rápido progresso nas técnicas de aprendizado profundo, impulsionado pelo aumento da capacidade de computação, grandes conjuntos de dados e algoritmos de treinamento eficazes.

ESTRUTURA DE UMA RNAs

As RNAs consistem em unidades de processamento chamadas de "neurônios artificiais" ou "nós", organizadas em camadas interconectadas. **Existem três tipos principais de camadas em uma RNA:**

- **Camada de Entrada (Input Layer):** Esta camada recebe os dados de entrada e transmite-os para a camada seguinte. Cada neurônio na camada de entrada corresponde a uma característica ou atributo do dado de entrada.
- **Camadas Ocultas (Hidden Layers):** Estas são camadas intermediárias entre a camada de entrada e a camada de saída. Cada camada oculta contém vários neurônios que realizam cálculos complexos para aprender padrões nos dados.
- **Camada de Saída (Output Layer):** Esta camada fornece a resposta ou saída final da RNA com base nas informações processadas pelas camadas anteriores. O número de neurônios nesta camada depende do tipo de problema, como classificação (um neurônio para cada classe) ou regressão (um neurônio para um valor contínuo).

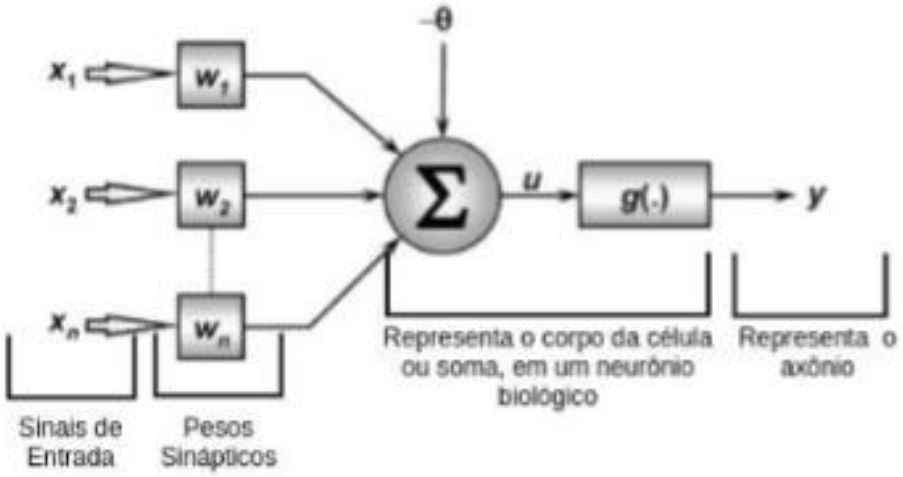
ESTRUTURA DE UMA RNAs

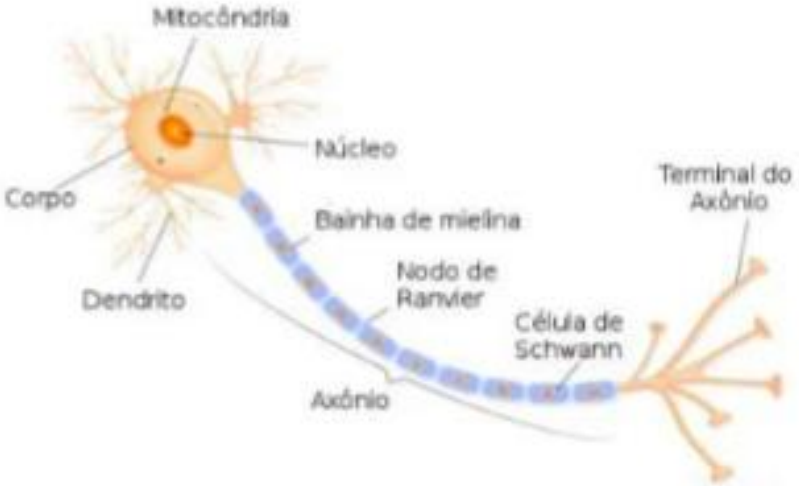


https://**www.researchgate.net/figure/Figura-1-Rede-Neural-Artificial-Multicamadas_fig2_240772105**

ESTRUTURA DE UMA RNAs

As RNAs consistem em unidades de processamento chamadas de "neurônios artificiais" ou "nós", organizadas em camadas interconectadas.





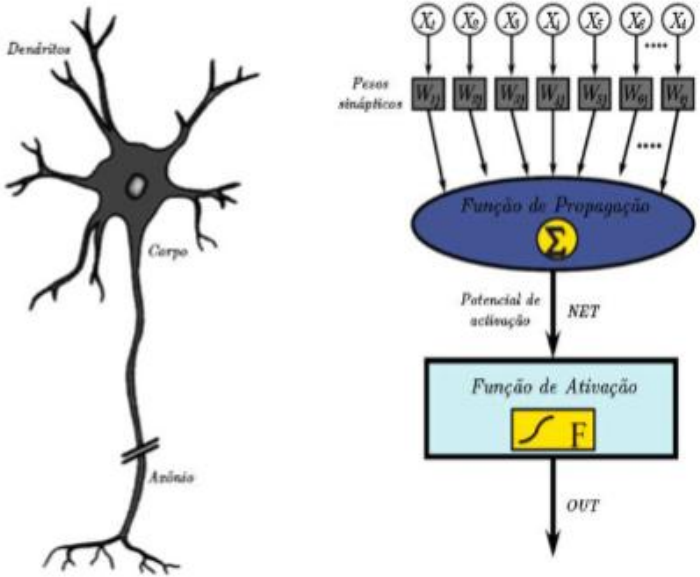
https://www.researchgate.net/figure/Figura-19-Neuronio-biologico-esquerda-e-neuronio-artificial-direita_fig18_300015903

ESTRUTURA DE UMA RNAs

Função de Propagação (Inputs): Cada neurônio artificial recebe um conjunto de entradas. Cada entrada é multiplicada por um peso associado a essa entrada. A ponderação das entradas reflete a importância relativa de cada entrada para o neurônio.

Função de Ativação (Activation Function): Após a soma ponderada das entradas, o neurônio aplica uma função de ativação. Essa função determina se o neurônio deve "disparar“ (produzir uma saída) ou permanecer inativo. A função de ativação introduz não linearidade na operação do neurônio.

Saída (Output): A saída do neurônio é o resultado da aplicação da função de ativação à soma ponderada das entradas. A saída pode ser enviada como entrada para outros neurônios em redes neurais mais complexas.



https://www.researchgate.net/figure/Figura-19-Neuronio-biologico-esquerda-e-neuronio-artificial-direita_fig18_300015903

TIPOS DE RNAs

Perceptron Simples (Single-Layer Perceptron):

Aplicações: Classificação linear de dados, separação de padrões lineares simples.

Redes Neurais Feedforward Multicamadas (Multilayer Perceptron - MLP):

Aplicações: Classificação, regressão, reconhecimento de padrões, processamento de linguagem natural, visão computacional, aprendizado profundo.

Redes Neurais Convolucionais (Convolutional Neural Networks - CNN):

Aplicações: Processamento de imagens, reconhecimento de objetos em imagens, classificação de imagens, visão computacional.

Redes Neurais Recorrentes (Recurrent Neural Networks - RNN):

Aplicações: Processamento de sequências temporais, análise de texto, tradução automática, previsão de séries temporais.

TIPOS DE RNAs

Redes Neurais Longa de Curta Memória (Long Short-Term Memory - LSTM):

Aplicações: Sequências temporais de modelagem, processamento de linguagem natural, tradução automática, análise de áudio.

Redes Generativas Adversariais (Generative Adversarial Networks - GAN):

Aplicações: Geração de imagens e vídeos realistas, tradução de estilo de arte, aprimoramento de imagens, criação de conteúdo gerado por máquina.

Redes Neurais Recorrentes Bidirecionais (Bidirectional RNN):

Aplicações: Processamento de sequências temporais, tradução automática, análise de sentimento, reconhecimento de fala.

Redes Neurais Autoencoder (Autoencoder):

Aplicações: Redução de dimensionalidade, reconstrução de dados, detecção de anomalias, compressão de dados.

TIPOS DE RNAs

Redes Neurais Siamesas (Siamese Networks):

Aplicações: Comparação de pares de dados, reconhecimento de padrões em imagens, verificação de identidade.

Redes Neurais Residuais (Residual Neural Networks - ResNet):

Aplicações: Classificação de imagens, visão computacional, tarefas de reconhecimento de padrões.

Redes Neurais de Memória de Atenção (Attention Mechanism):

Aplicações: Processamento de linguagem natural, tradução automática, sistemas de pergunta e resposta, resumo de texto.

PERCEPTRON E NEURÔNIO ARTIFICIAL

O **Perceptron** é um modelo de neurônio artificial específico que foi proposto por Frank Rosenblatt em 1957. É um modelo muito simples e pode ser visto como uma versão inicial e limitada dos neurônios artificiais. O Perceptron é projetado para resolver problemas de classificação linear, onde o objetivo é separar dados em duas classes com base em um limite de decisão linear. Ele usa uma função de ativação degrau (step function) para produzir uma saída binária (0 ou 1) com base na comparação da soma ponderada das entradas com um limite. Os Perceptrons não podem resolver problemas que não sejam linearmente separáveis, o que limita suas capacidades.

PERCEPTRON E NEURÔNIO ARTIFICIAL

Neurônio Artificial

O neurônio artificial é uma unidade genérica de processamento em uma rede neural artificial (RNA). É uma abstração matemática mais geral que pode ser adaptada para várias tarefas e arquiteturas de RNAs. Os neurônios artificiais podem usar diferentes funções de ativação, não apenas a função degrau. Funções de ativação comuns incluem sigmoid, ReLU (Rectified Linear Unit), tangente hiperbólica e outras. Os neurônios artificiais podem ser usados em várias arquiteturas de RNAs, incluindo redes neurais feedforward multicamadas (MLP), redes neurais convolucionais (CNN), redes neurais recorrentes (RNN) e outras. Eles são mais flexíveis do que Perceptrons individuais e podem resolver uma ampla gama de problemas, incluindo aqueles que não são linearmente separáveis.

PERCEPTRON SIMPLES

```python
import numpy as np

# Função de ativação - Função degrau
def step_function(x):
    return 1 if x > 0 else 0

# Perceptron (neurônio artificial)
def perceptron(input_data, weights):
    # Calcula a soma ponderada das entradas e pesos
    weighted_sum = np.dot(input_data, weights)
    # Aplica a função de ativação
    output = step_function(weighted_sum)
    return output

# Pesos sinápticos (pesos do perceptron)
weights = np.array([0.5, 0.5])  # Inicialização aleatória

# Dados de entrada
input_data = np.array([0.2, 0.3])

# Calcula a saída do perceptron
output = perceptron(input_data, weights)

# Exibe a saída
print("Saída do perceptron:", output)
```

ONDE PODEMOS UTILIZAR?

Porta lógica AND: Você pode usar esse perceptron para implementar uma porta lógica AND. Os valores de entrada seriam os bits (0 ou 1) que você deseja operar, e a saída seria 1 apenas quando ambos os bits de entrada forem 1.

Porta lógica OR: Da mesma forma, você pode usá-lo para implementar uma porta lógica OR. A saída seria 1 se pelo menos um dos bits de entrada for 1.

Classificação de pontos em um gráfico: Suponha que você tenha pontos em um plano bidimensional e deseje criar um limite de decisão para separar os pontos em duas classes. O perceptron poderia ser treinado para tomar essa decisão, com base nas coordenadas dos pontos como entradas.

Detecção de spam em e-mails: Você pode representar características de e-mails, como palavras-chave ou estatísticas de texto, como entradas e treinar o perceptron para classificar e-mails como spam ou não spam com base nesses recursos.

Classificação de imagens em preto e branco: Se você tiver imagens em preto e branco com pixels representados como valores binários (0 para preto, 1 para branco), você poderia usar esse perceptron para tarefas simples de reconhecimento de padrões.

PERCEPTRON

O Perceptron é a unidade básica de uma rede neural. Ele recebe uma entrada, aplica uma transformação linear (combinação ponderada) e passa o resultado por uma função de ativação para produzir uma saída. Matematicamente, a saída de um Perceptron pode ser descrita como:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

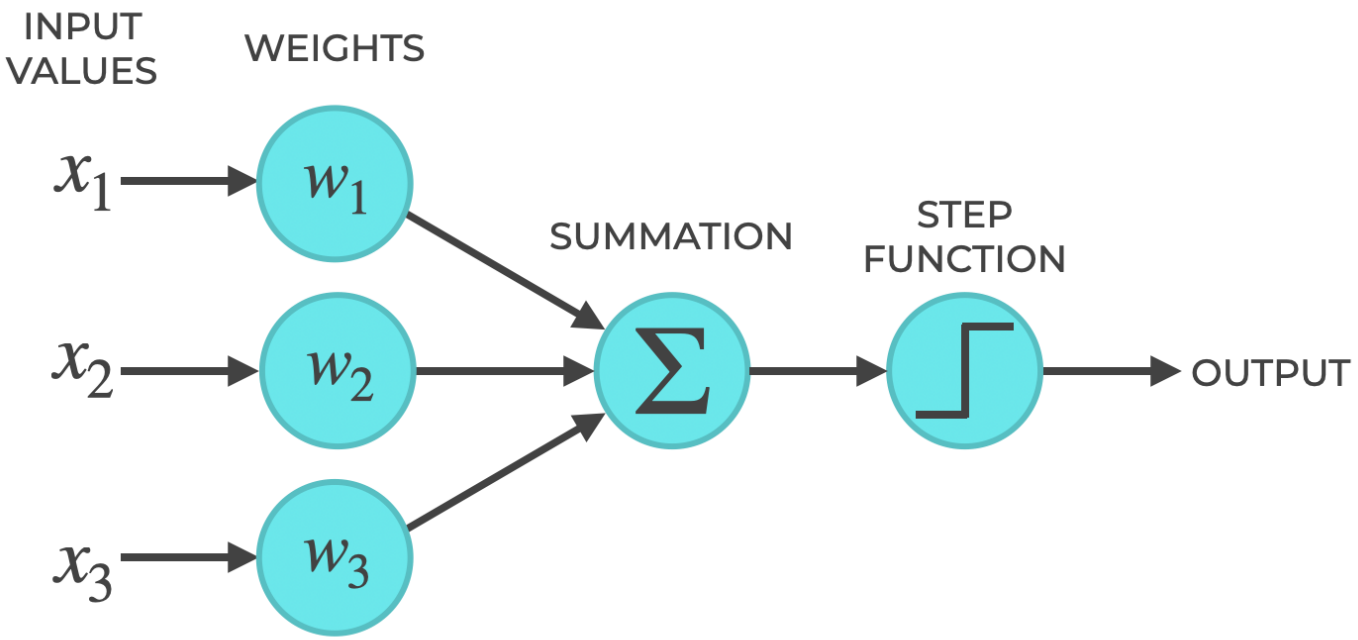
Onde:

- x_i são as entradas,
- w_i são os pesos,
- b é o viés (bias),
- f é a função de ativação.

O Perceptron é capaz de aprender ajustando os pesos e o viés com base em um conjunto de dados de treinamento, utilizando algoritmos como o gradiente descendente.

PERCEPTRON

THE STRUCTURE OF A PERCEPTRON



PERCEPTRON

Funcionamento:

Entradas: Recebem diversos valores numéricos, representando os dados a serem processados.

Pesos: Cada entrada possui um peso associado, que determina a força da sua influência na saída do neurônio.

Função de soma ponderada: Combina as entradas multiplicadas por seus respectivos pesos, gerando um valor único.

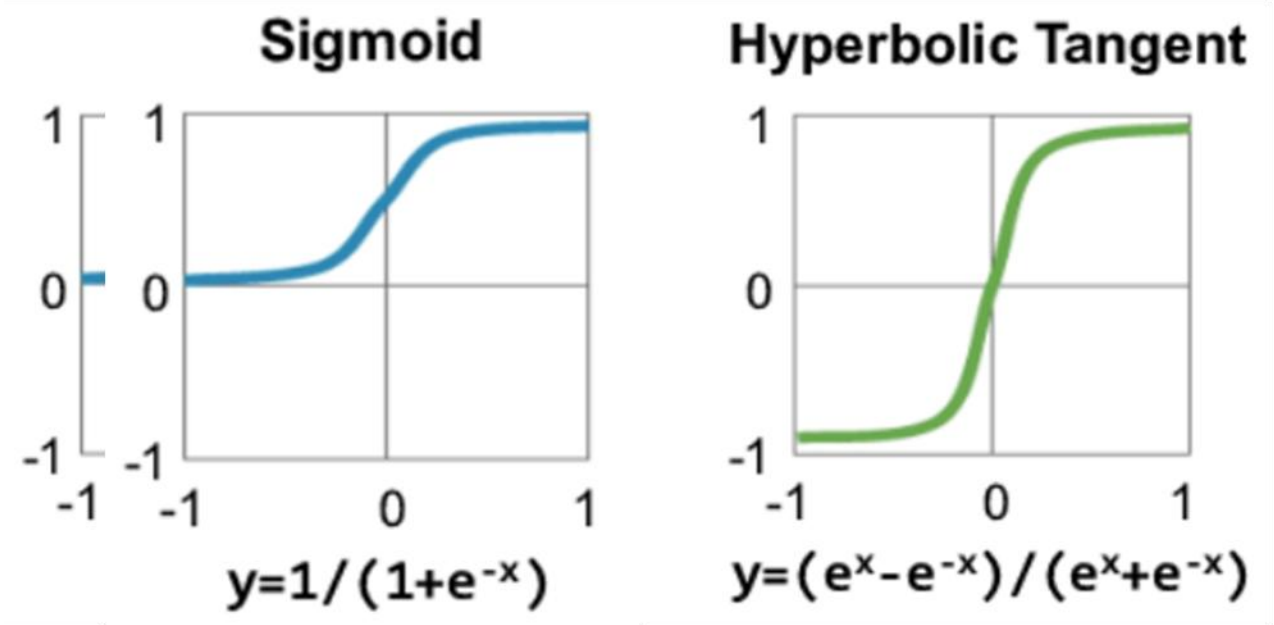
Função de ativação: Aplica uma função matemática à soma ponderada, determinando a saída final do neurônio. Essa função pode ser binária (0 ou 1) ou complexa, gerando valores entre 0 e 1.

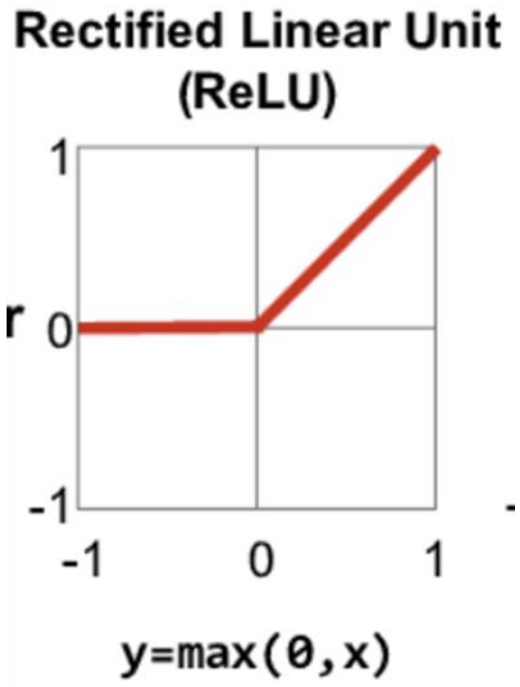
Axônio: Envia a saída do neurônio para outros neurônios da rede.

PERCEPTRON

Função de Ativação	Cálculo Matemático	Descrição	Sugestão de Uso
Step (Degrau)	$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$	Ativa o neurônio apenas quando a entrada ultrapassa um limiar (0 ou 1).	Perceptron Simples, problemas lineares binários.
Sigmoide	$f(x) = \frac{1}{1+e^{-x}}$	Suaviza a saída para um intervalo entre 0 e 1, interpretável como probabilidade.	Classificação binária, última camada em redes neurais para previsões probabilísticas.
Tanh (Tangente Hiperbólica)	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	Similar à sigmoide, mas retorna valores entre -1 e 1, melhor para redes neurais profundas.	Camadas ocultas de redes neurais onde se deseja saídas centralizadas em torno de zero.
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	Mantém apenas valores positivos, acelera o treinamento e reduz o problema do desvanecimento do gradiente.	Camadas ocultas em redes neurais profundas (CNNs, RNNs, etc.).
Leaky ReLU	$f(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$	Similar à ReLU, mas permite pequenos valores negativos para evitar neurônios mortos.	Redes neurais profundas onde a ReLU convencional causa problemas com "neurônios mortos".
Softmax	$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	Converte valores em probabilidades para múltiplas classes, garantindo que a soma seja 1.	Última camada em problemas de classificação multiclasse.
Swish	$f(x) = x \cdot \text{sigmoid}(x)$	Aprimora a suavização do gradiente, permitindo valores negativos úteis.	Alternativa moderna ao ReLU em redes neurais profundas.

PERCEPTRON


Sigmoid
1
0
-1
-1
0
1
y=1/(1+e^-x)
Hyperbolic Tangent
1
0
-1
-1
0
1
y=(e^x-e^-x)/(e^x+e^-x)


Rectified Linear Unit
(ReLU)
1
0
-1
-1
0
1
y=max(0,x)

PERCEPTRON

Contexto:

Você é um cientista de dados contratado por uma operadora de telecomunicações para ajudar a reduzir a taxa de **churn** (cancelamento de assinaturas). A empresa possui uma base de dados de clientes com informações demográficas, padrões de uso dos serviços e histórico de pagamentos. O objetivo é identificar clientes que têm alta probabilidade de cancelar seus planos, para que a empresa possa tomar ações proativas, como oferecer descontos ou planos personalizados, e assim reduzir a taxa de churn.

PERCEPTRON

Churn é uma métrica que indica a percentagem de clientes que cancelam ou não renovam um serviço ou produto em um determinado período. É também conhecido **como abandono de clientes** ou **rotatividade**. O churn é uma métrica importante para empresas, principalmente aquelas com modelos baseados em assinatura.

$$\text{CHURN RATE} = \frac{\text{Nº de clientes cancelados no mês}}{\text{Nº de clientes no início do mês}} \times 100$$

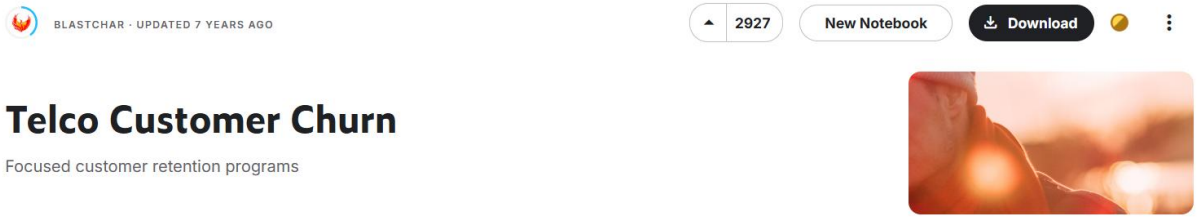
HUANG, Bingquan; KECHADI, Mohand Tahar; BUCKLEY, Brian. Customer churn prediction in telecommunications. Expert Systems with Applications, v. 39, n. 1, p. 1414-1425, 2012.

PERCEPTRON

Base de Dados:

A base de dados utilizada será a "Telco Customer Churn", disponível no Kaggle. Este dataset contém informações de 7.043 clientes de uma operadora de telecomunicações fictícia, com 21 variáveis, incluindo:

- Variáveis demográficas: Gênero, idade, se possui parceiro(a), se possui dependentes.
- Variáveis de serviço: Tipo de contrato, tipo de internet, serviços adicionais (como streaming, segurança online, etc.).
- Variáveis de pagamento: Método de pagamento, valor mensal, se houve atraso no pagamento.
- Variável alvo: Churn (Sim/Não) - indica se o cliente cancelou o serviço.



https://www.kaggle.com/datasets/blastchar/telco-customer-churn

PERCEPTRON

Telco-Customer-Churn.csv

1 to 10 of 7043 entries Filter

customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	No	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	No	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.3	1840.75	No
9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	No	No	No	No	No	Month-to-month	Yes	Electronic check	70.7	151.65	Yes
9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	No	No	Yes	No	Yes	Yes	Month-to-month	Yes	Electronic check	99.65	820.5	Yes
1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No	Yes	No	No	Yes	No	Month-to-month	Yes	Credit card (automatic)	89.1	1949.4	No
6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	Yes	No	No	No	No	No	Month-to-month	No	Mailed check	29.75	301.9	No
7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	No	No	Yes	Yes	Yes	Yes	Month-to-month	Yes	Electronic check	104.8	3046.05	Yes
6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Yes	Yes	No	No	No	No	One year	No	Bank transfer (automatic)	56.15	3487.95	No

Show 10 per page

1 2 10 100 600 700 705

BLASTCHAR · UPDATED 7 YEARS AGO

2927 New Notebook Download

Telco Customer Churn

Focused customer retention programs

https://www.kaggle.com/datasets/blastchar/telco-customer-churn

PERCEPTRON

1) Pré-processamento dos Dados:

Tratar valores faltantes (missing values).

Codificar variáveis categóricas (como gênero, tipo de contrato) em valores numéricos.

Normalizar as variáveis numéricas (como valor mensal) para garantir que estejam na mesma escala.

2) Divisão dos Dados:

Dividir a base em conjuntos de treinamento (70%) e teste (30%).

3) Implementação do Perceptron:

Criar um Perceptron Simples com uma função de ativação degrau ou sigmóide.

Treinar o modelo utilizando o algoritmo de descida do gradiente.

PERCEPTRON

4) Avaliação do Modelo:

Calcular métricas de desempenho, como acurácia, precisão, recall e F1-score.

Acurácia: Porcentagem de previsões corretas.

Precisão: Proporção de verdadeiros positivos entre todos os positivos previstos.

Recall: Proporção de verdadeiros positivos entre todos os positivos reais.

F1-Score: Média harmônica entre precisão e recall.

Analisar a matriz de confusão para entender os erros de classificação.

5) Interpretação dos Resultados:

Identificar quais variáveis têm maior impacto na previsão de churn.

Sugerir ações para a empresa com base nos clientes identificados como propensos ao churn.

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Carregar os dados
df = pd.read_csv('Telco-Customer-Churn.csv')

# Selecionar as colunas relevantes
features = ['MonthlyCharges', 'Contract', 'OnlineSecurity', 'Churn']
df = df[features]

# Tratar valores faltantes
df = df.dropna()

# Codificação de variáveis categóricas
label_encoders = {}
for col in ['Contract', 'OnlineSecurity', 'Churn']:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Separar variáveis independentes e dependente
X = df[['MonthlyCharges', 'Contract', 'OnlineSecurity']].values
y = df['Churn'].values

# Normalizar os dados
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Divisão entre treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

```

```python

# Implementação do Perceptron
class Perceptron:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def activation(self, x):
        return 1 if x >= 0 else 0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.epochs):
            for idx, x_i in enumerate(X):
                linear_output = np.dot(x_i, self.weights) + self.bias
                y_pred = self.activation(linear_output)

                # Atualização dos pesos
                update = self.lr * (y[idx] - y_pred)
                self.weights += update * x_i
                self.bias += update

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return np.array([self.activation(x) for x in linear_output])

# Treinar o modelo
perceptron = Perceptron(lr=0.01, epochs=1000)
perceptron.fit(X_train, y_train)

y_pred = perceptron.predict(X_test)

```

```python
# Avaliação do modelo
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Acurácia: {accuracy:.4f}")
print("Matriz de Confusão:\n", conf_matrix)
print("Relatório de Classificação:\n", report)

```

```
Acurácia: 0.7132
Matriz de Confusão:
 [[1342  197]
 [ 409  165]]
Relatório de Classificação:
               precision    recall  f1-score   support

           0       0.77      0.87      0.82      1539
           1       0.46      0.29      0.35       574

    accuracy                           0.71      2113
   macro avg       0.61      0.58      0.58      2113
weighted avg       0.68      0.71      0.69      2113
```

O resultado mostra que a acurácia do modelo foi de 71,32%, mas há um grande desequilíbrio na capacidade do Perceptron de prever corretamente a classe "1" (clientes que cancelam o serviço). O recall para essa classe está muito baixo (0.29), indicando que o modelo está deixando de prever corretamente muitos clientes propensos ao churn.

PERCEPTRON

Ajustar os Pesos Inicialmente;

Aumentar o Número de Épocas e Ajustar a Taxa de Aprendizado;

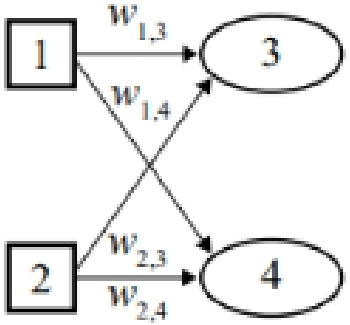
Alterar a Função de Ativação;

Usar Mais Features;

Testar um Perceptron de Múltiplas Camadas (MLP).

REDES NEURAIS DE CAMADA ÚNICA

As redes neurais de camada única são uma estrutura fundamental com todas as suas entradas conectadas diretamente com somente uma saída, você pode ver uma rede de camada única simples de duas entradas com duas interligações.



x_1	x_2	$f(x)_1$	$f(x)_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

REDES NEURAIS DE CAMADA ÚNICA

Exemplo de Rede Neural Perceptron em Python para Classificação de Flores

Cenário: Imagine um jardim com três tipos de flores: íris, lírios e rosas. Cada flor possui características distintas, como tamanho da pétala, cor e formato. Desejamos criar um sistema que possa **classificar automaticamente** novas flores com base em suas características.

Entrada: Três valores numéricos representando as características da flor (tamanho da pétala, cor e formato);

Pesos: Três valores numéricos que determinam a importância de cada característica na classificação.

Função de soma ponderada: Soma os produtos de cada entrada por seu peso respectivo.

Função de ativação: Determina se a flor pertence à classe "íris", "lírio" ou "rosa".

EXEMPLO

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

# Função de ativação Softmax
def softmax(x):
    exp_x = np.exp(x - np.max(x))  # Subtrai o máximo para evitar overflow numérico
    return exp_x / exp_x.sum(axis=0)

# Treinamento do perceptron para múltiplas classes
def treinar_perceptron(entradas, saidas, taxa_aprendizagem, epocas):
    num_classes = len(set(saidas))  # Determina a quantidade de classes
    pesos = np.random.rand(entradas.shape[1], num_classes)

    for epoca in range(epocas):
        for i in range(len(entradas)):
            # Calcular soma ponderada
            soma_ponderada = np.dot(entradas[i], pesos)

            # Calcular saída do perceptron com softmax
            probabilidades = softmax(soma_ponderada)

            # Criar vetor esperado de saída one-hot
            y_real = np.zeros(num_classes)
            y_real[saidas[i]] = 1

            # Ajustar pesos com base no erro
            erro = y_real - probabilidades
            pesos += taxa_aprendizagem * np.outer(entradas[i], erro)

    return pesos
```

```python

# Exemplo de uso
dados_entrada = np.array([
    [5.1, 3.5, 1.4],  # Íris
    [4.9, 3.0, 1.4],  # Íris
    [5.1, 3.6, 1.4],  # Íris
    [6.4, 3.0, 1.7],  # Lírio
    [6.3, 3.3, 1.5],  # Lírio
    [5.7, 3.0, 1.3],  # Rosa
    [5.8, 2.7, 1.3],  # Rosa
])

rotulos_saida = np.array([0, 0, 0, 1, 1, 2, 2])

# Normalizar os dados
scaler = StandardScaler()
dados_entrada = scaler.fit_transform(dados_entrada)

taxa_aprendizagem = 0.1
epocas = 300

# Treinar o perceptron
pesos_treinados = treinar_perceptron(dados_entrada, rotulos_saida, taxa_aprendizagem, epocas)

# Classificar nova flor
nova_flor = np.array([[5.8, 2.5, 1.0]])
nova_flor = scaler.transform(nova_flor)
soma_ponderada = np.dot(nova_flor, pesos_treinados)
probabilidades = softmax(soma_ponderada[0])
saida = np.argmax(probabilidades)

if saida == 0:
    print("Nova flor classificada como Íris")
elif saida == 1:
    print("Nova flor classificada como Lírio")
else:
    print("Nova flor classificada como Rosa")
```

EXEMPLO

- **Preparação dos Dados:** Coletar e organizar os dados, a dividir os dados em conjuntos de treinamento e teste.
- **Desenvolvimento do Modelo:** Definir a função de ativação, a implementar a função de treinamento do Perceptron.
- **Treinamento do Modelo:** Ajustar os pesos do Perceptron com base nos dados de treinamento.
- **Avaliação do Modelo:** Validar a precisão do modelo utilizando os dados de teste.
- **Classificação de Novos Exemplos:** Usar o modelo treinado para classificar novas amostras.
- **Visualização dos Resultados:** Plotar os dados para verificar visualmente as classificações.

REDES MULTICAMADAS

As **redes multicamadas (MLP)** são uma extensão do **Perceptron**, compostas por várias camadas de neurônios interconectados. Essas camadas incluem:

- **Camada de entrada:** Recebe os dados brutos.
- **Camadas ocultas:** Realizam transformações não lineares nos dados.
- **Camada de saída:** Produz o resultado final.

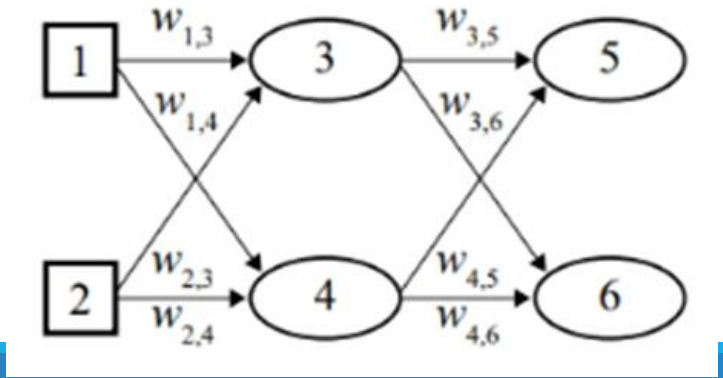
A principal vantagem das MLPs é sua capacidade de aprender representações hierárquicas dos dados, o que as torna poderosas para tarefas complexas.

REDES MULTICAMADAS

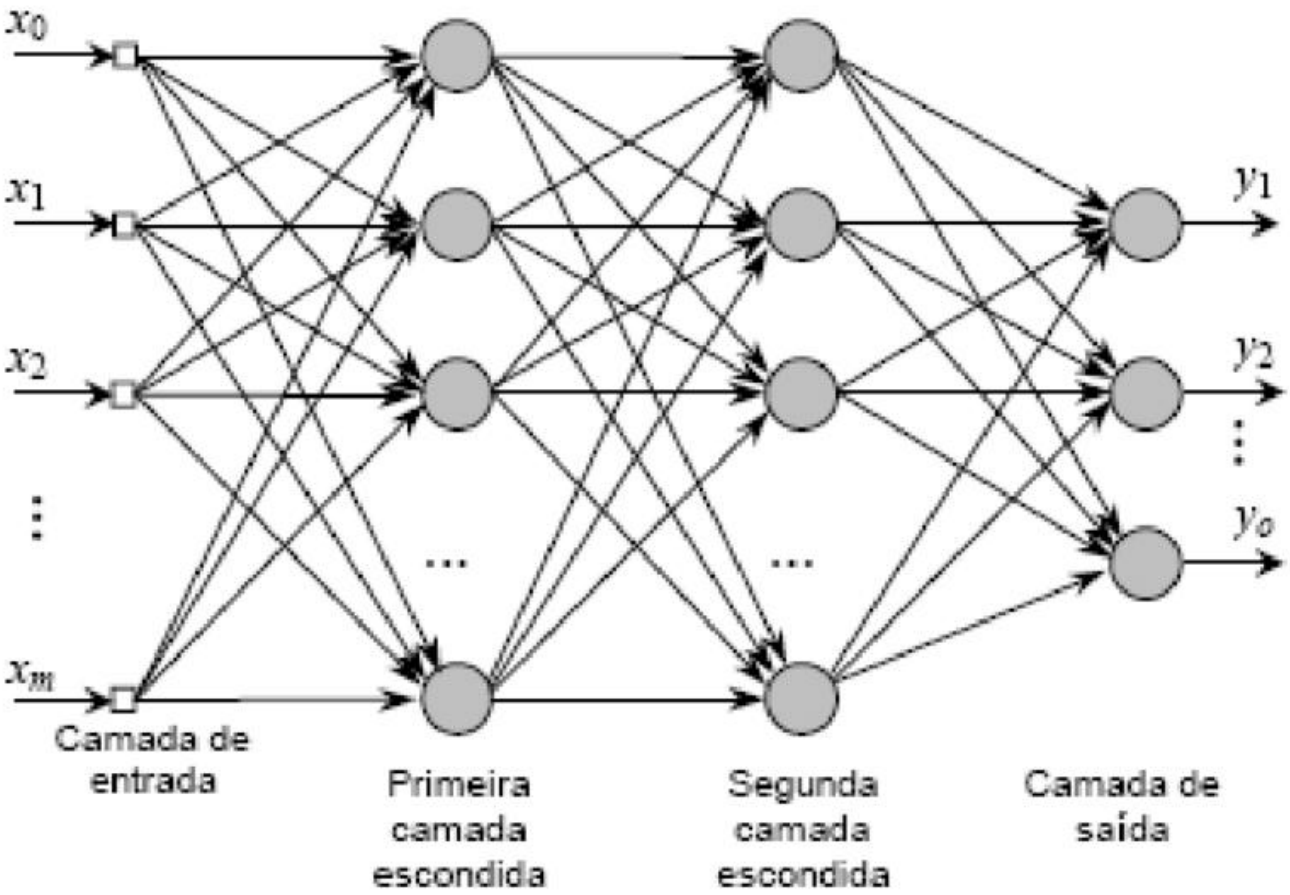
As **redes multicamadas (MLP)** são uma extensão do **Perceptron**, compostas por várias camadas de neurônios interconectados. Essas camadas incluem:

- **Camada de entrada:** Recebe os dados brutos.
- **Camadas ocultas:** Realizam transformações não lineares nos dados.
- **Camada de saída:** Produz o resultado final.

A principal vantagem das MLPs é sua capacidade de aprender representações hierárquicas dos dados, o que as torna poderosas para tarefas complexas.



REDES MULTICAMADAS



EXEMPLO

Problema: Classificação de Dígitos Manuscritos em Processamento de Cheques Bancários

Vamos considerar um cenário em que um banco deseja automatizar o processo de leitura e verificação de cheques manuscritos. Tradicionalmente, a verificação manual dos dígitos nos cheques é um processo demorado e suscetível a erros humanos. Para aumentar a eficiência e a precisão, o banco decide implementar uma solução baseada em aprendizado de máquina para classificar automaticamente os dígitos manuscritos.

Objetivo: Desenvolver um sistema capaz de reconhecer e classificar dígitos manuscritos nos cheques, para que a leitura dos valores seja automatizada e precisa.

Solução: Utilizar uma Perceptron de Múltiplas Camadas (MLP) treinada com um conjunto de dados de dígitos manuscritos para classificar os números escritos nos cheques.

EXEMPLO

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Carregar o dataset de dígitos
digits = load_digits()
X = digits.data
y = digits.target

# Dividir os dados em conjuntos de treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalizar os dados
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Criar e treinar a MLP
mlp = MLPClassifier(hidden_layer_sizes=(50,), max_iter=1000, random_state=42)
mlp.fit(X_train, y_train)

# Fazer previsões no conjunto de teste
y_pred = mlp.predict(X_test)
```

```python
# Fazer previsões no conjunto de teste
y_pred = mlp.predict(X_test)

# Avaliar o modelo
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Visualizar alguns resultados
def plot_digits(X, y_true, y_pred=None):
    fig, axes = plt.subplots(4, 4, figsize=(10, 10))
    for i, ax in enumerate(axes.flat):
        ax.imshow(X[i].reshape(8, 8), cmap='gray')
        if y_pred is not None:
            ax.set_title(f'True: {y_true[i]}\nPred: {y_pred[i]}')
        else:
            ax.set_title(f'True: {y_true[i]}')
        ax.axis('off')
    plt.show()

# Mostrar alguns dígitos com suas previsões
plot_digits(X_test, y_test, y_pred)
```

EXEMPLO

Avaliação do Modelo: Após o treinamento, o modelo é testado com o conjunto de teste para avaliar sua precisão. Métricas de classificação, como o relatório de classificação e a matriz de confusão, são utilizadas para analisar o desempenho do modelo.

Implementação no Sistema do Banco: Uma vez validado, o modelo é integrado ao sistema de processamento de cheques do banco. Quando um cheque é digitalizado, as imagens dos dígitos são passadas pelo modelo para serem classificadas automaticamente.

EXEMPLO

Problema: Classificação Automática de Roupas em um E-commerce

Vamos considerar um cenário em que uma grande plataforma de e-commerce deseja melhorar a experiência do usuário e a eficiência do catálogo de produtos. A plataforma recebe milhões de imagens de roupas enviadas por vendedores diariamente. Classificar essas roupas manualmente é um processo demorado e propenso a erros. Para resolver esse problema, a plataforma decide implementar uma solução de aprendizado de máquina para classificar automaticamente as roupas nas categorias apropriadas.

Objetivo: Desenvolver um sistema capaz de classificar imagens de roupas enviadas para a plataforma de e-commerce em uma das dez categorias: T-shirt/top, Calça, Pullover, Vestido, Casaco, Sandália, Camisa, Tênis, Bolsa e Bota.

Solução: Utilizar uma rede neural profunda treinada com o conjunto de dados Fashion MNIST para classificar automaticamente as imagens de roupas nas categorias apropriadas.

EXEMPLO

Coleta de Dados: A plataforma de e-commerce coleta um grande conjunto de dados de imagens de roupas, que são etiquetadas manualmente com a categoria correta. Essas imagens são usadas para treinar e validar o modelo de classificação.

Pré-processamento dos Dados: As imagens são normalizadas para que os valores dos pixels estejam no intervalo [0, 1], facilitando o treinamento da rede neural.

Desenvolvimento do Modelo: A rede neural é definida com uma camada de entrada que achata a imagem em um vetor unidimensional, uma camada oculta com 128 neurônios e função de ativação ReLU, e uma camada de saída com 10 neurônios e função de ativação softmax para classificar as imagens em uma das dez categorias.

Treinamento do Modelo: O modelo é treinado utilizando os dados de treino. Durante o treinamento, o modelo ajusta seus pesos para minimizar a perda, medida pela entropia cruzada categórica.

EXEMPLO

Avaliação do Modelo: O modelo é testado com o conjunto de dados de teste para avaliar sua precisão. Métricas de avaliação, como a acurácia, são utilizadas para medir o desempenho do modelo.

Implementação no Sistema de E-commerce: Uma vez validado, o modelo é integrado ao sistema de e-commerce. Quando um vendedor faz o upload de uma imagem de roupa, o modelo classifica automaticamente a imagem na categoria apropriada, facilitando a navegação e a busca de produtos para os usuários.

EXEMPLO

```python
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# Carregar o conjunto de dados Fashion MNIST
fashion_mnist = keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

# Definir os nomes das classes
class_names = ['T-shirt/top', 'Calça', 'Pullover', 'Vestido', 'Casaco',
               'Sandália', 'Camisa', 'Tênis', 'Bolsa', 'Bota']

# Pré-processamento dos dados
train_images = train_images / 255.0
test_images = test_images / 255.0

# Definir a arquitetura da rede neural
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),  # Camada de entrada: achatamento da imagem em um vetor unidimensional
    keras.layers.Dense(128, activation='relu'),  # Camada oculta com 128 neurônios e função de ativação ReLU
    keras.layers.Dense(10, activation='softmax')  # Camada de saída com 10 neurônios (um para cada classe) e função de ativação softma
])

# Compilar o modelo
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```python
# Compilar o modelo
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Treinar o modelo
model.fit(train_images, train_labels, epochs=5)
# Avaliar o modelo com os dados de teste
test_loss, test_acc = model.evaluate(test_images, test_labels)
print('\nAcurácia do teste:', test_acc)
# Fazer previsões com o modelo treinado
predictions = model.predict(test_images)

# Função auxiliar para mostrar a imagem e a previsão correspondente
def plot_image(i, predictions_array, true_label, img):
    predictions_array, true_label, img = predictions_array[i], true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])

    plt.imshow(img, cmap=plt.cm.binary)

    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'

    plt.xlabel("{} {:2.0f}% ({})".format(class_names[predicted_label],
                                         100 * np.max(predictions_array),
                                         class_names[true_label]),
                                         color=color)
```

```python
# Função auxiliar para mostrar o valor predito em formato de gráfico de barras
def plot_value_array(i, predictions_array, true_label):
    predictions_array, true_label = predictions_array[i], true_label[i]
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)

    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')

# Visualizar algumas previsões
num_rows = 5
num_cols = 3
num_images = num_rows * num_cols
plt.figure(figsize=(2 * 2 * num_cols, 2 * num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2 * num_cols, 2 * i + 1)
    plot_image(i, predictions, test_labels, test_images)
    plt.subplot(num_rows, 2 * num_cols, 2 * i + 2)
    plot_value_array(i, predictions, test_labels)
plt.show()
```

DÚVIDAS ou PERGUNTAS?

MUITO OBRIGADO!!!!

daniel.ohata@facens.br

REFERÊNCIAS

BISONG, Ekaba; BISONG, Ekaba. Google colaboratory. **Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners**, p. 59-64, 2019.

DE ANDRADE, Amanda Figueiredo et al. A ÉTICA NO USO DA INTELIGÊNCIA ARTIFICIAL E SEUS RISCOS JURÍDICOS. Revista Acadêmica Online, v. 11, n. 56, p. e1400-e1400, 2025.

ISZCZUK, Ana Claudia Duarte et al. Evoluções das tecnologias da indústria 4.0: dificuldades e oportunidades para as micro e pequenas empresas. **Brazilian Journal of Development**, v. 7, n. 5, p. 50614-50637, 2021.

MARCONI, Francesco. Artificial Intelligence Backbone. < https://twitter.com/fpmarconi/status/794208040207740928 >, 2016 Acesso 01/01/2019.

MICHALSKI, Ryszard Stanislaw; CARBONELL, Jaime Guillermo; MITCHELL, Tom M. (Ed.). **Machine learning: An artificial intelligence approach**. Springer Science & Business Media, 2013.

ROSÁRIO, João Maurício. **Automação industrial**. Editora Baraúna, 2012.

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. Malaysia; Pearson Education Limited, 2016.

SANTOS, Gustavo Soares. Novas Tecnologias Aplicadas na Construção Civil: Conceitos da Indústria 4.0. RCT-Revista de Ciência e Tecnologia, v. 8, 2022.

ZHOU, Zhi-Hua. **Machine learning**. Springer nature, 2021.